自适应多尺度决策系统设计文档

> **版本**: v1.0
> **创建日期**: 2026-03-20
> **状态**: 设计阶段

目录

1. 项目背景与动机
2. 问题分析
3. 需求分析
4. 系统架构设计
5. 方案对比分析
6. 核心组件详解
7. 实施路线图
8. 附录

---

1. 项目背景与动机

1.1 当前系统现状

PredictionRTS 项目已实现基于 Decision Transformer 的行为预测系统，包含：

- **Decision Transformer (DT)**: 序列行为预测模型
- **Q-Network**: 动作价值评估网络
- **State Transition Predictor**: 状态转移预测器

1.2 评估结果差异

评估方式	准确率	评估目标
`run_decision_system.py`	**98.00%**	固定20步历史后预测第21步
`run_sc2_optimal_agent.py`	**29.38%**	完整轨迹的每一步预测

1.3 准确率差异的根本原因

```
run_decision_system.py 评估方式:
┌──────────────────────────────────────────────────────────┐
│ 轨迹: [0, 1, 2, ..., 19, 20]                              │
│                          ↑                                 │
│                      预测这里                              │
│               (有完整的20步历史)                           │
└──────────────────────────────────────────────────────────┘
```

```
run_sc2_optimal_agent.py 评估方式:
┌──────────────────────────────────────────────────────────┐
| 轨迹: [0, 1, 2, ..., 17]                                 |
|         ↑  ↑  ↑  ...  ↑                                  |
|         预测每一步（历史从0步开始动态增长）                |
|                                                          |
|  Step 0: 无历史 → 预测                                   |
|  Step 5: 5步历史 → 预测                                  |
|  Step 10: 10步历史 → 预测                                |
└──────────────────────────────────────────────────────────┘
```

核心问题:

1. DT 模型在 context_window=20 下训练，对短历史预测能力差
2. 早期决策（历史不足10步）时，模型表现显著下降
3. 缺乏根据历史长度自适应调整策略的能力

1.4 设计动机

构建一个**自适应多尺度决策系统**，具备以下核心能力:

能力	描述
历史自适应	根据可用历史长度自动选择最优决策策略
多尺度预测	支持单步和多步预测，根据场景选择最优预测跨度
动态策略切换	在不同阶段（初始/早期/稳定）使用不同决策策略
置信度评估	综合评估决策可靠性，支持策略回退

---

2. 问题分析

2.1 不同历史长度下的挑战

阶段	可用历史	主要挑战	当前方案问题
初始时刻	0-2步	几乎无历史信息	DT 无法做出有意义的预测
早期阶段	3-10步	历史不足 context_window	填充0导致信息失真
稳定阶段	11-20步	接近训练条件	性能逐渐恢复
成熟阶段	20+步	历史充足	可考虑多步预测优化

2.2 数据特征分析

```
数据统计:
├── 轨迹长度: min=13, max=25, mean=18
├── 动作分布: b(22%), d(19%), a(17%), c(15%), j(11%), ...
├── 状态数量: 909个唯一状态 / 71860总状态 (1.3% 唯一)
└── 游戏结果: Win/Loss 标签可通过最终状态判断
```

2.3 现有模型局限性

模型	局限性
DT (ctx=20)	短历史时表现差；固定窗口限制灵活性
Q-Network	仅提供单步价值，无序列信息
State Predictor	准确率仅49%，状态空间过大

---

3. 需求分析

3.1 功能需求

FR-1: 历史自适应决策

```
IF 可用历史 < 5步:
    使用 Q值最大化策略
ELIF 可用历史 < 10步:
    使用 DT_ctx5 + Q验证
ELIF 可用历史 < 20步:
    使用 DT_ctx10 + Hybrid
ELSE:
    使用 DT_ctx20 + 多步预测
```

FR-2: 多步预测能力

```
支持预测步数: 1, 3, 5

选择逻辑:
  高置信度 (>0.8) → 可预测更多步
  中置信度 (0.5-0.8) → 单步预测
  低置信度 (<0.5) → 单步 + Q值验证
```

FR-3: 模型池管理

```
模型池包含:
├── Q-Network (共享)
├── State Predictor (共享)
├── DT_ctx5 (短历史专用)
├── DT_ctx10 (中等历史专用)
└── DT_ctx20 (长历史专用)
```

FR-4: 策略路由

```
输入: 可用历史长度, 当前状态, 决策上下文
输出: 最优模型, 决策策略, 预测步数
```

3.2 非功能需求

NFR	描述	指标
性能	决策延迟	< 50ms/决策
可扩展性	模型池扩展	支持动态添加新模型
可维护性	配置驱动	所有策略可通过配置修改
可观测性	决策日志	记录每次决策的完整信息

3.3 核心能力矩阵

能力	初始时刻	早期阶段	稳定阶段	成熟阶段
Q值决策	✓ 主用	✓ 备用	✓ 验证	✓ 验证
DT预测	✗ 不可用	△ 受限	✓ 可用	✓ 主用
多步预测	✗ 不可用	✗ 不可用	△ 受限	✓ 可用
状态预测	✗ 不可用	△ 参考用	✓ 可用	✓ 可用

---

4. 系统架构设计

4.1 整体架构

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              自适应多尺度决策系统                                  │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│    输入层                                                                     │
│    ┌──────────────────────────────────────────────────────────────────────┐   │
│    │  当前状态 + 历史序列 + 奖励序列 + 可用历史长度                              │   │
```

```
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                   │                                            │
│                                   ▼                                            │
│   路由层                                                                       │
│   ┌──────────────────────────────────────────────────────────────────────┐   │
│   │                    Strategy Router (策略路由器)                        │   │
│   │                                                                        │   │
│   │   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐             │   │
│   │   │ 历史分析器   │──▶│ 策略选择器   │──▶│ 置信度评估   │             │   │
│   │   └──────────────┘   └──────────────┘   └──────────────┘             │   │
│   │                                                                        │   │
│   │   输出: (selected_model, strategy, prediction_steps)                  │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                   │                                            │
│                                   ▼                                            │
│   模型层                                                                       │
│   ┌──────────────────────────────────────────────────────────────────────┐   │
│   │                      Model Pool (模型池)                              │   │
│   │                                                                        │   │
│   │   ┌──────────────────────────────────────────────────────────────┐   │   │
│   │   │                   Decision Transformers                      │   │   │
│   │   │  ┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐        │   │   │
│   │   │  │DT_ctx5  │  │DT_ctx10 │  │DT_ctx15 │  │DT_ctx20 │        │   │   │
│   │   │  │pred:1,3 │  │pred:1,3 │  │pred:1,3,│  │pred:1,3,│        │   │   │
│   │   │  │         │  │         │  │   5     │  │   5     │        │   │   │
│   │   │  └─────────┘  └─────────┘  └─────────┘  └─────────┘        │   │   │
│   │   └──────────────────────────────────────────────────────────────┘   │   │
│   │                                                                        │   │
│   │   ┌──────────────────┐  ┌──────────────────┐                        │   │
│   │   │    Q-Network     │  │  State Predictor  │  (共享模型)            │   │
│   │   │  (动作价值评估)  │  │  (状态转移预测)   │                        │   │
│   │   └──────────────────┘  └──────────────────┘                        │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                   │                                            │
│                                   ▼                                            │
│   决策层                                                                       │
│   ┌──────────────────────────────────────────────────────────────────────┐   │
│   │                   Adaptive Decision Agent                             │   │
│   │                                                                        │   │
│   │   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐             │   │
│   │   │ 动作选择器   │──▶│ 多步预测器   │──▶│ 结果整合器   │             │   │
│   │   └──────────────┘   └──────────────┘   └──────────────┘             │   │
│   │                                                                        │   │
│   │   输出: (actions, confidence, decision_info)                          │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                   │                                            │
│                                   ▼                                            │
│   输出层                                                                       │
│   ┌──────────────────────────────────────────────────────────────────────┐   │
│   │  最优动作序列 + 决策置信度 + 使用的策略信息 + 详细决策日志            │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                                                                │
└──────────────────────────────────────────────────────────────────────────────┘
```

4.2 组件交互流程

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              决策请求处理流程                                  │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  1. 接收决策请求                                                              │
│     │                                                                        │
│     │  Input: (current_state, history, available_length)                     │
│     │                                                                        │
│     ▼                                                                        │
│  2. 策略路由                                                                  │
│     │                                                                        │
│     ├─▶ 分析可用历史长度                                                      │
│     ├─▶ 选择合适的模型窗口                                                    │
│     ├─▶ 确定决策策略 (q_only/dt_only/hybrid)                                  │
│     └─▶ 确定预测步数                                                          │
│     │                                                                        │
│     ▼                                                                        │
│  3. 模型推理                                                                  │
│     │                                                                        │
│     ├─▶ 从模型池加载选定模型                                                  │
│     ├─▶ 准备模型输入 (填充/截断历史)                                          │
│     ├─▶ 执行前向推理                                                          │
│     └─▶ 获取预测结果和概率分布                                                │
│     │                                                                        │
│     ▼                                                                        │
│  4. 价值评估 (如果使用 hybrid 策略)                                           │
│     │                                                                        │
│     ├─▶ Q-Network 评估候选动作价值                                            │
│     ├─▶ 结合 DT 概率和 Q 值                                                   │
│     └─▶ 选择综合得分最高的动作                                                │
│     │                                                                        │
│     ▼                                                                        │
│  5. 多步预测 (如果启用)                                                       │
│     │                                                                        │
│     ├─▶ 使用 State Predictor 预测未来状态                                     │
│     ├─▶ 自回归生成动作序列                                                    │
│     └─▶ 计算累积置信度                                                        │
│     │                                                                        │
│     ▼                                                                        │
│  6. 置信度评估                                                                │
│     │                                                                        │
│     ├─▶ 综合评估决策置信度                                                    │
│     ├─▶ 如果置信度过低，触发策略回退                                          │
│     └─▶ 记录决策信息                                                          │
│     │                                                                        │
│     ▼                                                                        │
│  7. 返回决策结果                                                              │
│                                                                              │
│     Output: (actions, confidence, decision_info)                             │
│                                                                              │
```

```
└──────────────────────────────────────────────────────────────────────────────────────┘
```

4.3 模型池设计

```
MODEL_POOL_CONFIG = {
    # 共享模型
    'q_network': {
        'type': 'shared',
        'path': 'cache/model/q_network.pth',
        'description': '动作价值评估网络',
    },
    'state_predictor': {
        'type': 'shared',
        'path': 'cache/model/state_predictor.pth',
        'description': '状态转移预测器',
    },

    # DT 模型池
    'DT_ctx5': {
        'type': 'decision_transformer',
        'path': 'cache/model/dt_ctx5.pth',
        'context_window': 5,
        'min_history': 5,
        'max_history': 9,
        'prediction_steps': [1, 3],
        'strategy': 'hybrid',
        'fallback': 'q_only',
    },
    'DT_ctx10': {
        'type': 'decision_transformer',
        'path': 'cache/model/dt_ctx10.pth',
        'context_window': 10,
        'min_history': 10,
        'max_history': 19,
        'prediction_steps': [1, 3, 5],
        'strategy': 'hybrid',
        'fallback': 'DT_ctx5',
    },
    'DT_ctx20': {
        'type': 'decision_transformer',
        'path': 'cache/model/best_model.pth',
        'context_window': 20,
        'min_history': 20,
        'max_history': float('inf'),
        'prediction_steps': [1, 3, 5],
        'strategy': 'hybrid',
        'fallback': 'DT_ctx10',
    },
}
```

4.4 策略路由规则

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              策略路由决策树                                │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│                        ┌──────────────────┐                              │
│                        │ 可用历史长度 H   │                              │
│                        └──────────┬───────┘                              │
│                                   │                                      │
│                  ┌────────────────┼────────────────┐                     │
│                  │                │                │                     │
│                  ▼                ▼                ▼                     │
│            H < 5          5 ≤ H < 20        H ≥ 20                       │
│                  │                │                │                     │
│                  ▼                │                │                     │
│          ┌────────────────┐       │                │                     │
│          │   Q-Only       │       │                │                     │
│          │   策略         │       │                │                     │
│          └────────────────┘       │                │                     │
│                                   │                │                     │
│                  ┌────────────────┼────────┐       │                     │
│                  │                │        │       │                     │
│                  ▼                        ▼       │                     │
│            5 ≤ H < 10            10 ≤ H < 20      │                     │
│                  │                        │       │                     │
│                  ▼                        ▼       │                     │
│          ┌────────────────┐       ┌────────────────┐                     │
│          │   DT_ctx5      │       │   DT_ctx10     │                     │
│          │   Hybrid       │       │   Hybrid       │                     │
│          │   pred: 1,3    │       │   pred: 1,3,5  │                     │
│          └────────────────┘       └────────────────┘                     │
│                                                         │                │
│                                                         ▼                │
│                                              ┌────────────────┐          │
│                                              │   DT_ctx20     │          │
│                                              │   Hybrid       │          │
│                                              │   pred: 1,3,5  │          │
│                                              └────────────────┘          │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

---

5. 方案对比分析

5.1 方案 A: 完整多模型方案

设计描述

为每个 context_window 和 prediction_step 组合训练独立模型。

模型配置

模型	Context Window	Prediction Steps	用途
DT_ctx1_pred1	1	1	初始时刻（备选）
DT_ctx5_pred1	5	1	早期单步
DT_ctx5_pred3	5	3	早期多步
DT_ctx10_pred1	10	1	中期单步
DT_ctx10_pred5	10	5	中期多步
DT_ctx20_pred1	20	1	成熟期单步
DT_ctx20_pred5	20	5	成熟期多步

优势

- ✓ 每个模型针对特定场景优化，性能最优
- ✓ 精确匹配历史长度和预测需求
- ✓ 可针对性调参

劣势

- ✗ 训练时间长（~2小时）
- ✗ 模型文件多（~50MB/模型，共7-8个）
- ✗ 内存占用大
- ✗ 维护成本高

适用场景

追求极致性能，资源充足的生产环境

5.2 方案 B: 单模型动态处理方案

设计描述

只使用一个 DT_ctx20 模型，通过动态填充/截断适应不同历史长度。

实现逻辑

```
def adaptive_forward(model, states, available_history):
    if available_history >= 20:
        return model(states[-20:])
    elif available_history >= 10:
        # 填充到 20
        padded = pad_to_length(states, 20)
        return model(padded)
    else:
        # 历史太短，返回 None 触发 Q-only
        return None
```

优势

- ✓ 无需额外训练
- ✓ 实现简单
- ✓ 维护成本低
- ✓ 内存占用小

劣势

- ✗ 短历史时性能下降明显
- ✗ 填充导致信息失真
- ✗ 无法做多步预测优化

适用场景

快速原型验证，资源受限环境

5.3 方案 C: 混合方案（推荐）

设计描述

训练关键窗口的模型（ctx5, ctx10），复用已有 ctx20 模型，共享 Q-Network 和 State Predictor。

模型配置

模型	Context Window	Prediction Steps	状态
Q-Network	-	1	已有
State Predictor	-	-	已有
DT_ctx5	5	1, 3	新增
DT_ctx10	10	1, 3, 5	新增
DT_ctx20	20	1, 3, 5	已有

优势

- ✓ 平衡性能和训练成本
- ✓ 覆盖关键历史窗口
- ✓ 复用已有模型
- ✓ 训练时间可控（~45分钟）
- ✓ 维护成本适中

劣势

- △ 需要额外的模型管理逻辑
- △ 内存占用适中（新增2个模型）

适用场景

大多数生产环境（推荐）

5.4 方案对比总结

维度	方案 A (完整)	方案 B (单模型)	方案 C (混合)
训练时间	~2小时	0分钟	~45分钟
模型数量	7-8个	1个	3个
磁盘占用	~400MB	~50MB	~150MB
内存占用	高	低	中
实现复杂度	高	低	中
性能	最优	较差	良好
维护成本	高	低	中
灵活性	最高	最低	高

---

6. 核心组件详解

6.1 模型池管理器 (Model Pool Manager)

职责

- 管理所有模型的加载、缓存和访问
- 提供统一的模型获取接口
- 支持模型的懒加载和预加载

接口设计

```python
class ModelPoolManager:
    def __init__(self, config: Dict):
        """
        初始化模型池

        Args:
            config: 模型池配置
        """
        pass

    def get_model(self, model_key: str) -> torch.nn.Module:
        """
        获取指定模型（懒加载）

        Args:
            model_key: 模型标识（如 'DT_ctx5'）

        Returns:
            模型实例
        """
        pass

    def select_model_for_history(self, available_history: int) -> str:
        """
        根据可用历史选择最优模型

        Args:
            available_history: 可用历史步数

        Returns:
            模型标识
        """
        pass

    def preload_all(self):
        """预加载所有模型到内存"""
        pass

    def clear_cache(self):
        """清除模型缓存，释放内存"""
        pass
```

配置示例

```yaml
# configs/model_pool.yaml
models:
  DT_ctx5:
    type: decision_transformer
    path: cache/model/dt_ctx5.pth
    context_window: 5
```

```yaml
    min_history: 5
    max_history: 9

  DT_ctx10:
    type: decision_transformer
    path: cache/model/dt_ctx10.pth
    context_window: 10
    min_history: 10
    max_history: 19

  DT_ctx20:
    type: decision_transformer
    path: cache/model/best_model.pth
    context_window: 20
    min_history: 20
    max_history: 1000

shared_models:
  q_network:
    path: cache/model/q_network.pth
  state_predictor:
    path: cache/model/state_predictor.pth
```

6.2 策略路由器 (Strategy Router)

职责

- 分析当前决策上下文
- 选择最优决策策略
- 确定预测步数
- 处理策略回退

接口设计

```python
class StrategyRouter:
    def __init__(self, model_pool: ModelPoolManager, config: Dict):
        pass

    def route(
        self,
        available_history: int,
        current_state: int,
        context: Dict
    ) -> RoutingDecision:
        """
        路由决策

        Args:
            available_history: 可用历史步数
            current_state: 当前状态
```

```
            context: 决策上下文

        Returns:
            RoutingDecision: 包含模型、策略、预测步数的决策
        """
        pass

    def should_fallback(
        self,
        confidence: float,
        model_key: str
    ) -> Tuple[bool, str]:
        """
        判断是否需要回退到备选策略

        Args:
            confidence: 当前置信度
            model_key: 当前模型

        Returns:
            (should_fallback, fallback_model)
        """
        pass

@dataclass
class RoutingDecision:
    model_key: str           # 选择的模型
    strategy: str            # 决策策略 (q_only/dt_only/hybrid)
    prediction_steps: int    # 预测步数
    fallback_model: str      # 备选模型
    confidence_threshold: float  # 置信度阈值
```

路由规则配置

```yaml
# configs/strategy_router.yaml
routing_rules:
  - min_history: 0
    max_history: 4
    strategy: q_only
    model: null
    prediction_steps: 1

  - min_history: 5
    max_history: 9
    strategy: hybrid
    model: DT_ctx5
    prediction_steps: [1, 3]
    fallback: q_only

  - min_history: 10
    max_history: 19
    strategy: hybrid
```

```yaml
    model: DT_ctx10
    prediction_steps: [1, 3, 5]
    fallback: DT_ctx5

  - min_history: 20
    max_history: 1000
    strategy: hybrid
    model: DT_ctx20
    prediction_steps: [1, 3, 5]
    fallback: DT_ctx10

confidence_thresholds:
  multi_step: 0.8     # 高于此才用多步预测
  single_step: 0.5    # 高于此才用单步预测
  fallback: 0.3       # 低于此触发回退
```

6.3 置信度评估器 (Confidence Estimator)

职责

- 综合评估决策置信度
- 整合多个置信度来源
- 支持置信度校准

置信度来源

来源	权重	说明
DT 概率	0.4	DT 输出的最高概率
Q 一致性	0.2	DT 选择与 Q 最优是否一致
历史充分度	0.2	可用历史 / context_window
历史验证	0.2	该模型在验证集上的表现

接口设计

```python
class ConfidenceEstimator:
    def compute_confidence(
        self,
        dt_probs: List[float],
        q_values: Dict[int, float],
        available_history: int,
        model_key: str,
        validation_metrics: Dict
    ) -> float:
        """
        计算综合置信度

        Args:
```

```
            dt_probs: DT 输出的概率分布
            q_values: Q-Network 输出的价值
            available_history: 可用历史步数
            model_key: 使用的模型
            validation_metrics: 模型验证指标

        Returns:
            置信度 [0, 1]
        """
        pass
```

6.4 多步预测器 (Multi-Step Predictor)

职责

- 实现多步预测逻辑
- 管理预测误差累积
- 提供预测置信度衰减

预测方式对比

方式	实现复杂度	准确度	误差累积
自回归	低	中	有
多输出	高	高	无
混合	中	高	少

接口设计

```
class MultiStepPredictor:
    def predict(
        self,
        model: torch.nn.Module,
        history: Dict,
        steps: int,
        method: str = 'autoregressive'
    ) -> MultiStepPrediction:
        """
        多步预测

        Args:
            model: DT 模型
            history: 历史序列
            steps: 预测步数
            method: 预测方法

        Returns:
            MultiStepPrediction: 包含动作序列和置信度
```

```
        """
        pass

@dataclass
class MultiStepPrediction:
    actions: List[int]           # 预测的动作序列
    confidences: List[float]     # 每步的置信度
    states: List[int]            # 预测的状态序列（如果使用 State Predictor）
    cumulative_confidence: float # 累积置信度
```

6.5 自适应决策 Agent (Adaptive Decision Agent)

职责

- 整合所有组件
- 提供统一的决策接口
- 管理决策日志

接口设计

```
class AdaptiveDecisionAgent:
    def __init__(
        self,
        model_pool: ModelPoolManager,
        strategy_router: StrategyRouter,
        confidence_estimator: ConfidenceEstimator,
        multi_step_predictor: MultiStepPredictor,
        device: torch.device
    ):
        pass

    def get_action(
        self,
        current_state: int,
        history: Dict,
        last_reward: float = None
    ) -> DecisionResult:
        """
        获取最优动作

        Args:
            current_state: 当前状态
            history: 历史信息
            last_reward: 上一步奖励

        Returns:
            DecisionResult: 决策结果
        """
        pass

    def reset(self):
```

```
        """重置 Agent 状态"""
        pass

@dataclass
class DecisionResult:
    action: int                  # 选择的动作
    action_name: str             # 动作名称
    actions: List[int]           # 动作序列（多步预测时）
    confidence: float            # 决策置信度
    model_used: str              # 使用的模型
    strategy_used: str           # 使用的策略
    prediction_steps: int        # 预测步数
    decision_info: Dict          # 详细决策信息
```

---

7. 实施路线图

7.1 阶段划分

阶段 1: 文档与数据准备（30分钟）

任务	文件	预计时间
创建系统设计文档	`doc/ADAPTIVE_DECISION_SYSTEM.md`	20分钟
实现数据准备工具	`src/utils/data_utils.py`	10分钟

阶段 2: 模型训练（45分钟）

任务	命令	预计时间
训练 DT_ctx5	`python scripts/run_train_dt.py --ctx 5 --epochs 20`	20分钟
训练 DT_ctx10	`python scripts/run_train_dt.py --ctx 10 --epochs 20`	20分钟
验证所有模型	`python scripts/run_validate_models.py`	5分钟

阶段 3: 核心组件实现（60分钟）

任务	文件	预计时间
实现模型池管理器	`src/decision/model_pool.py`	20分钟
实现策略路由器	`src/decision/strategy_router.py`	20分钟
实现多步预测器	`src/decision/multi_step_predictor.py`	20分钟

阶段 4: Agent 集成与测试（60分钟）

任务	文件	预计时间
创建 AdaptiveDecisionAgent	`src/env/agents/AdaptiveDecisionAgent.py`	25分钟
创建评估脚本	`scripts/run_adaptive_agent.py`	15分钟
对比测试所有模式	-	20分钟

7.2 时间估算

```
┌──────────────────────────────────────────────────────────────────┐
│                          实施时间线                                │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  阶段 1: 文档与数据准备 ═══════════════▒▒▒
```

里程碑	完成标准	预计时间
M3: 组件完成	核心组件单元测试通过	T+135min
M4: 系统集成	Adaptive Agent 功能测试通过	T+180min
M5: 验收	所有决策模式对比测试完成	T+195min

---

8. 附录

8.1 代码示例

使用模型池

```
from src.decision.model_pool import ModelPoolManager

# 初始化模型池
config = load_config('configs/model_pool.yaml')
model_pool = ModelPoolManager(config)

# 预加载所有模型
model_pool.preload_all()

# 根据历史选择模型
model_key = model_pool.select_model_for_history(available_history=12)
model = model_pool.get_model(model_key)
```

使用策略路由器

```
from src.decision.strategy_router import StrategyRouter

router = StrategyRouter(model_pool, config)

# 路由决策
decision = router.route(
    available_history=12,
    current_state=state_id,
    context={'rtg': target_return}
)

print(f"选择模型: {decision.model_key}")
print(f"决策策略: {decision.strategy}")
print(f"预测步数: {decision.prediction_steps}")
```

使用自适应 Agent

```
from src.env.agents.AdaptiveDecisionAgent import AdaptiveDecisionAgent

agent = AdaptiveDecisionAgent(
    model_pool=model_pool,
```

```
    strategy_router=router,
    confidence_estimator=conf_estimator,
    multi_step_predictor=predictor,
    device=device
)

# 获取决策
result = agent.get_action(
    current_state=state_id,
    history={
        'states': states_history,
        'actions': actions_history,
        'rtgs': rtgs_history
    }
)

print(f"动作: {result.action_name}")
print(f"置信度: {result.confidence:.2f}")
print(f"使用模型: {result.model_used}")
```

8.2 配置模板

模型池配置

```yaml
# configs/model_pool.yaml
version: 1.0

models:
  DT_ctx5:
    type: decision_transformer
    path: cache/model/dt_ctx5.pth
    context_window: 5
    min_history: 5
    max_history: 9
    prediction_steps: [1, 3]
    strategy: hybrid
    fallback: q_only
    preload: false

  DT_ctx10:
    type: decision_transformer
    path: cache/model/dt_ctx10.pth
    context_window: 10
    min_history: 10
    max_history: 19
    prediction_steps: [1, 3, 5]
    strategy: hybrid
    fallback: DT_ctx5
    preload: false

  DT_ctx20:
    type: decision_transformer
```

```yaml
    path: cache/model/best_model.pth
    context_window: 20
    min_history: 20
    max_history: 1000
    prediction_steps: [1, 3, 5]
    strategy: hybrid
    fallback: DT_ctx10
    preload: true

shared_models:
  q_network:
    path: cache/model/q_network.pth
    preload: true

  state_predictor:
    path: cache/model/state_predictor.pth
    preload: false
```

策略路由配置

```yaml
# configs/strategy_router.yaml
version: 1.0

routing_rules:
  - name: initial_phase
    min_history: 0
    max_history: 4
    strategy: q_only
    model: null
    prediction_steps: 1
    description: "初始阶段，历史不足，使用Q值"

  - name: early_phase
    min_history: 5
    max_history: 9
    strategy: hybrid
    model: DT_ctx5
    prediction_steps: [1, 3]
    fallback: q_only
    description: "早期阶段，使用短窗口DT"

  - name: mid_phase
    min_history: 10
    max_history: 19
    strategy: hybrid
    model: DT_ctx10
    prediction_steps: [1, 3, 5]
    fallback: DT_ctx5
    description: "中期阶段，使用中等窗口DT"

  - name: mature_phase
    min_history: 20
```

```yaml
    max_history: 1000
    strategy: hybrid
    model: DT_ctx20
    prediction_steps: [1, 3, 5]
    fallback: DT_ctx10
    description: "成熟阶段，使用标准窗口DT"

confidence_config:
  weights:
    dt_prob: 0.4
    q_consistency: 0.2
    history_score: 0.2
    validation_score: 0.2

  thresholds:
    multi_step: 0.8
    single_step: 0.5
    fallback: 0.3
```

8.3 性能指标

目标指标

指标	目标值	说明
整体准确率	> 60%	混合场景
短历史准确率	> 40%	0-10步历史
长历史准确率	> 80%	20+步历史
决策延迟	< 50ms	单次决策
内存占用	< 2GB	所有模型加载

8.4 参考资料

1. Decision Transformer: https://arxiv.org/abs/2106.01345
2. Offline Reinforcement Learning: https://arxiv.org/abs/2005.01643
3. Multi-Step Prediction in RL: https://arxiv.org/abs/1910.02141

文档结束